Introduction to Informatics and Programming
(CIT123001)

Florian Zuleger
WT 2026/27

Chair for Fundamentals of Programming
Technical University of Munich (TUM)

# Syntax of Programming Languages

Lecture 2

# Today

9. Syntax and semantics

10. Tokens and regular expressions

11. Grammar and syntax trees

# What is syntax?

**Syntax**: formal description of a language's words and sentences.

**Natural languages**
Words combine into a sentence.

`The` `cat` `sleeps` `.`

**A sentence.**

**Programming languages**
Precise, machine-checkable rules determine legal programs.

```
int x ;
```

**A variable declaration.**

# Syntax in natural languages

1. **Dictionaries:** which words are in use.
2. **Spelling and boundaries:** how words are written and separated.
3. **Grammar:** how words combine into sentences.
4. **Exceptions and intuition:** help judge unfamiliar forms.

The cat sleeps .

**A sentence.**

The sleeps cat .

**Same words, wrong order.**



# Four ingredients of a language specification

1. **Reserved words**: `if`, `int`, `else`, `while`.
2. **Token rules**: for example, which character sequences are names?
3. **Grammar rules**: how are tokens assembled into larger fragments?
4. **Context conditions**: for example, a variable must be declared before use.

# Four ingredients of a language specification

1. **Reserved words**: `if`, `int`, `else`, `while`.
2. **Token rules**: for example, which character sequences are names?
3. **Grammar rules**: how are tokens assembled into larger fragments?
4. **Context conditions**: for example, a variable must be declared before use.

**Reserved words**

`if` as a name? **No.**

**Token rules**

`x10`: **valid**; `10x`: **invalid**.

**Grammar**

`if (c) s; else t;` **valid**.

`if (c) else t;` **invalid**.

**Context**

`int x; ... write(x);` **yes**;
`write(y);` **no**.

# Syntax does not tell us what a program does

**Semantics** assigns meaning to a program.

- ↑ **Operational Semantics**: its possible executions.
- ↑ **Denotational Semantics**: the input–output mapping it computes.

**A syntactically legal program may still compute the wrong answer.**

⇒ **We must reason about both form and behavior.**

# Words with a fixed meaning

| | |
|---|---|
| `int` | name of a base type |
| `if`, `else`, `while` | reserved words in control structures |
| `(`, `)`, `"`, `'`, `{`, `}`, `,`, `;`, `_` | punctuation and other special characters |

A reserved word cannot also be used as a variable name.

# Admissible names: character classes

In the simplified lexical rule used here:

$$\textbf{letter} ::= \texttt{\$} \mid \texttt{_} \mid \texttt{a} \mid \cdots \mid \texttt{z} \mid \texttt{A} \mid \cdots \mid \texttt{Z}$$
$$\textbf{digit} ::= 0 \mid \cdots \mid 9$$

**Character classes** collect symbols that obey the same rule.

- The bar $\mid$ separates alternatives: choose one character.
- The dots $\cdots$ abbreviate omitted alternatives: $\texttt{a} \mid \cdots \mid \texttt{z}$ includes every lowercase letter from `a` to `z`.

**Remark:** This grammar is simplified. **Java** identifiers also permit many Unicode letters and follow additional lexical rules.

# Admissible names: ordering

$$\textbf{name} ::= \textbf{letter}\,(\textbf{letter} \mid \textbf{digit})^*$$

- First comes **one character** from **letter**.
- $^*$ means **repeat zero or more times**. Zero repetitions means leaving that part out.
- It is a **postfix operator**: it follows the part it repeats, here the whole group (**letter** | **digit**).
- On each repetition, choose a letter or a digit; the choices may differ.

$\Rightarrow$ **One letter, followed by any number of letters or digits.**

# Admissible names: examples

Apply the rule: one letter, then zero or more letters or digits.

| Matches | Does not match | Reason |
|---|---|---|
| `$178` | `5ABC` | First character is a digit. |
| `x` | `!Hello!` | `!` is neither a letter nor a digit. |
| `password` | `x'` | `'` is neither a letter nor a digit. |
| `_tmp` | `-178` | `-` is neither a letter nor a digit. |

# Admissible names: reserved words

**Matching the pattern is not enough.**

Reserved words such as `int` and `if` cannot be names.

In current **Java**, _ alone is also reserved.

# Integer constants

A constant is a value written directly in the program.

$$\textbf{number} ::= \textbf{digit}\ \textbf{digit}^*$$

The rule admits `17`, `12490`, `0`, and `70`. It does not describe `"Hello World!"` or `0.5e+128` as `int` constants.

**Remark:** A sign is a separate unary operator in our later expression grammar.
**Question:** How could we rule out leading zeros?

**In real Java, a leading zero means octal:** `010` has the value $8$.

# Regular expressions

**Regular expressions** describe sets of character sequences.

They are studied in their own right in ↑ **Automata Theory**.

### 1. Base cases

A literal character:

`a` `0` `$`

### 2. Construction

From regular expressions $R$ and $S$, form:

| | |
|---|---|
| $R \mid S$ | alternative: either $R$ or $S$ |
| $RS$ | concatenation: $R$ followed by $S$ |
| $R^*$ | repetition: zero or more occurrences of $R$ |
| $R?$ | option: zero or one occurrence of $R$ |

**Here: token** shapes. **Later:** the same notation in grammar rules.

# Regular expressions: examples

- (**letter letter**)*: an even number of letters, including zero.
- **letter*** `test` **letter***: words containing `test`.
- $ **digit*** `17`: a dollar sign, zero or more digits, then `17`.

↑ **Compiler Construction**

# Scanning and parsing

**First phase: ↑ Scanner**

**Identify**

- Reserved words
- Names
- Constants

**Remove**

- Whitespace
- Comments

⇒ **Match input against regular expressions to identify tokens.**

**Second phase: ↑ Parser**

Analyze how the tokens form the structure of the program.

# Programs consist of nested fragments

Our small language uses EBNF-style rules:

$$\begin{aligned} \textbf{program} &::= \textbf{decl}^{*}\ \textbf{stmt}^{*} \\ \textbf{decl} &::= \textbf{type}\ \textbf{name}\ (\texttt{,}\ \textbf{name})^{*}\ \texttt{;} \\ \textbf{type} &::= \texttt{int} \end{aligned}$$

- A program has zero or more declarations, then zero or more statements.
- A declaration gives one type to one or more comma-separated names.
- **For simplicity:** our only variable type here is `int`.

# The statement grammar: basic forms

$$
\begin{aligned}
\textbf{stmt} ::= \; & \texttt{;} \mid \texttt{\{}\ \textbf{stmt}^{*}\ \texttt{\}} \\
& \mid \textbf{name}\ \texttt{=}\ \textbf{expr}\ \texttt{;} \\
& \mid \textbf{name}\ \texttt{=}\ \texttt{readInt();} \\
& \mid \texttt{write(}\textbf{expr}\texttt{);}
\end{aligned}
$$

- **Empty statement:** ; does nothing.
- **Block:** braces enclose zero or more statements.
- **Assignment:** store an expression's value in a variable.
- **Input/output:** read an integer into a variable, or write an integer.

**Simplification:** string output from lecture 1 is omitted.

# The statement grammar: control flow

**Further alternatives for stmt:**

$$
\begin{aligned}
\textbf{stmt} ::= {} & \cdots \\
& \mid \texttt{if (}\textbf{cond}\texttt{)}\ \textbf{stmt} \\
& \mid \texttt{if (}\textbf{cond}\texttt{)}\ \textbf{stmt}\ \texttt{else}\ \textbf{stmt} \\
& \mid \texttt{while (}\textbf{cond}\texttt{)}\ \textbf{stmt}
\end{aligned}
$$

- **One-sided branch:** execute the statement only if the condition is true.
- **Two-sided branch:** choose one of two statements according to the condition.
- **Loop:** repeat the statement while the condition is true.

# Arithmetic expressions

$$\begin{aligned}\textbf{expr} ::= \; & \textbf{number} \mid \textbf{name} \mid (\textbf{expr}) \\ \mid \; & \textbf{unop expr} \\ \mid \; & \textbf{expr binop expr}\end{aligned}$$

- An expression can be a **constant**, a **variable**, or another expression enclosed in **parentheses**.
- A **unary operator** is applied to one expression.
- A **binary operator** is applied to two expressions.

$\Rightarrow$ **Arithmetic expressions in this small language yield integers.**

# Unary and binary operators

**unop** ::= `-`

**binop** ::= `+` | `-` | `*` | `/` | `%`

- The only **unary operator** here is arithmetic negation: `-x` negates the value of `x`.
- The **binary operators** are addition, subtraction, multiplication, integer division, and remainder.

**One operand**

`-x` uses **unop**.

**Two operands**

`x - y` uses **binop**.

# Boolean expressions (conditions)

$$
\begin{aligned}
\textbf{cond} &::= \texttt{true} \mid \texttt{false} \mid \texttt{(}\textbf{cond}\texttt{)} \\
&\quad \mid \textbf{expr comp expr} \\
&\quad \mid \textbf{bunop cond} \mid \textbf{cond bbinop cond} \\
\textbf{comp} &::= \texttt{==} \mid \texttt{!=} \mid \texttt{<} \mid \texttt{<=} \mid \texttt{>} \mid \texttt{>=} \\
\textbf{bunop} &::= \texttt{!} \\
\textbf{bbinop} &::= \texttt{\&\&} \mid \texttt{||}
\end{aligned}
$$

- Conditions yield **Boolean values**: `true` or `false`.
- Forms include constants, comparisons, and parenthesized conditions.
- Logical composition: `!` (not), `&&` (and), `||` (or).

# Grammar rules can be ambiguous

An ambiguous grammar needs **parsing conventions** to select one tree.

**Arithmetic expressions**
`x + y * z`

**Priorities (precedence)**
In **Java**, `*`, `/`, `%` bind before `+`, `-`.

**Associativity**
`x-y-z` → `(x-y)-z` (left).

**Conditional branches**
`if(c1) if(c2) s1 else s2`

**Convention for `else`**
In **Java**, it belongs to the **nearest unmatched `if`**.

# Make grouping explicit

**Avoid ambiguity:** make the intended grouping explicit with brackets.

**Parentheses `()`: expressions**

```
(x + y) * z
```

The addition is grouped before multiplication.

**Braces `{}`: statements**

```
if(c1) { if(c2) s1 }
else s2
```

Here, `else` belongs to the outer `if`.

# A syntax tree reveals the hierarchy

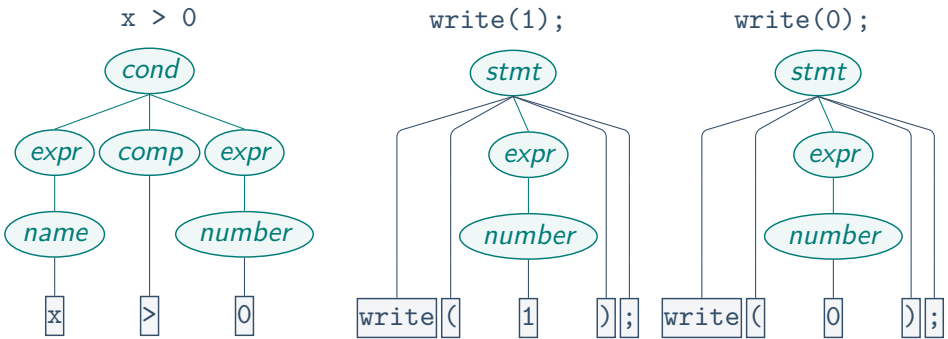
```
int x;
x = readInt();
if (x > 0) write(1);
else       write(0);
```

The ↑ **Parser** organizes tokens into a **syntax tree** using the grammar rules.

- **Teal ovals:** nonterminals, the names of program fragments.
- **Slate rectangles:** terminals, the concrete tokens at the leaves.
- **Teal edges** lead to fragments; **slate edges** lead to tokens.

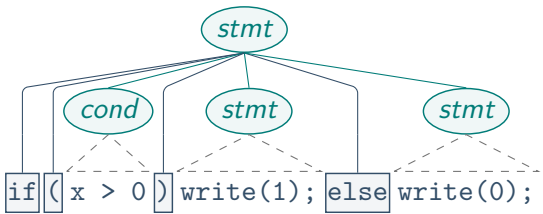
Start with small fragments, then combine their trees into a larger tree.

# Syntax trees of small fragments



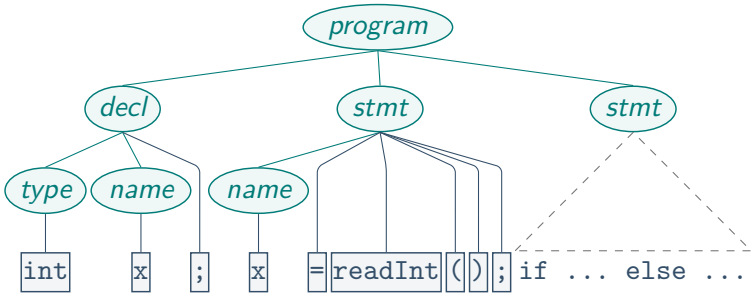

- **Rule applications:** each oval's children match a grammar alternative.
- **Token order:** read the rectangular leaves from left to right.

# Syntax tree of the `if` statement




- **Parsing:** the root uses the two-sided `if` grammar alternative.
- Its children follow token order: `if` `(`, a condition, `)`, a statement, `else`, another statement.
- **Dashed triangles** abbreviate the three subtrees from the previous slide.

# Syntax tree of the whole program




- **Program rule:** one declaration, then two statements.
- **Parsing:** a declaration, an input statement, then the `if` statement.
- **Dashed triangle:** expand the previous `if` tree in this position.

# EBNF: regular-expression notation in rules

**EBNF** (Extended Backus–Naur Form) is a collection of definitions of the form:

$\mathbf{A}_1 = \text{expression}_1$

$\mathbf{A}_2 = \text{expression}_2$

$\vdots$

**Left-hand side**

One name, such as $\mathbf{A}_1$ or **stmt**.

**Right-hand side**

Regular-expression notation, now combining **literals and names**.

A name may refer to its own rule: **recursion**.

**BNF** (Backus–Naur Form) is the special case using only sequences and alternatives, without the $*$ and ? shorthand.

**BNF, EBNF, context-free grammars: same expressive power.**

# Recursive rules: an inductive construction

**Example:** $\mathbf{A} ::= (\mathtt{a}\,\mathbf{A}\,\mathtt{b})?$

**1. Start with no characters**

The ? lets us omit the entire parenthesized part. The result is $\epsilon$, the **empty word**: zero characters, not even a space.

# Recursive rules: an inductive construction

**Example:** $\mathbf{A} ::= (\mathtt{a}\,\mathbf{A}\,\mathtt{b})?$

**1. Start with no characters**

The ? lets us omit the entire parenthesized part. The result is $\epsilon$, the **empty word**: zero characters, not even a space.

**2. Use a word we already know**

Insert this empty word into **A**: $\mathtt{a}\boxed{\epsilon}\mathtt{b} = \mathtt{ab}$.

The middle contributes no characters; only a and b remain.

# Recursive rules: an inductive construction

**Example:** $\mathbf{A} ::= (\mathtt{a}\,\mathbf{A}\,\mathtt{b})?$

**1. Start with no characters**

The ? lets us omit the entire parenthesized part. The result is $\epsilon$, the **empty word**: zero characters, not even a space.

**2. Use a word we already know**

Insert this empty word into **A**: $\mathtt{a}\boxed{\epsilon}\mathtt{b} = \mathtt{ab}$.

The middle contributes no characters; only a and b remain.

**3. Repeat the construction with the new word**

Now insert ab into **A**: $\mathtt{a}\boxed{\mathtt{ab}}\mathtt{b} = \mathtt{aabb}$.

# Regular expressions and recursive grammars

### Regular expressions

Describe token shapes, such as names and numbers.

**Repetition**, with independent counts:

$$\texttt{a}^*\texttt{b}^*$$

Allows `aab`, for example.

### Recursive grammar rules

Can also describe nested structures, such as expressions and statements.

Our rule **matches the counts**:

$$\{\epsilon, \texttt{ab}, \texttt{aabb}, \texttt{aaabbb}, \ldots\}$$

Adds one `a` at the front and one `b` at the end, each time.

**No regular expression can enforce these equal counts** without a fixed upper bound.

Recursive grammars are more expressive. ↑ **Automata Theory**

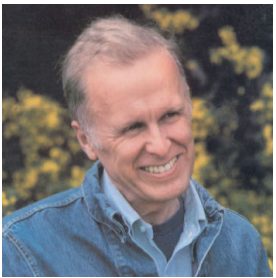
# Grammars and programming languages




Noam Chomsky, MIT

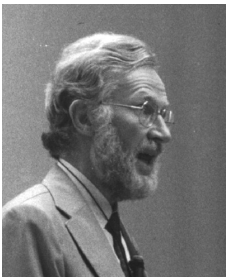
Chomsky hierarchy

↑ **Linguistics**



John Backus, IBM

Turing Award; Fortran



Peter Naur

Turing Award; Algol 60